

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

История PL/pgSQL

Структура блока, объявление переменных

Анонимные блоки

Подпрограммы на языке PL/pgSQL

Условные операторы и циклы

Вычисление выражений

История PL/pgSQL

Появился в версии 6.4 в 1998 году

- устанавливается по умолчанию с версии 9.0

Цели создания

- простой язык для написания пользовательских функций и триггеров
- добавить управляющие структуры к языку SQL
- сохранить возможность использования любых пользовательских типов, функций и операторов

Родословная: Oracle PL/SQL, Ада

PL/pgSQL — один из первых процедурных языков в PostgreSQL. Он впервые появился в 1998 году в версии 6.4, а с версии 9.0 стал устанавливаться по умолчанию при создании базы данных.

PL/pgSQL дополняет SQL, предоставляя такие возможности процедурных языков, как использование переменных и курсоров, условные операторы, циклы, обработка ошибок и т. д.

PL/pgSQL проектировался на основе языка Oracle PL/SQL, а тот, в свою очередь, был создан на основе подмножества языка Ада. Эта ветвь происходит от таких языков, как Алгол и Паскаль. Большинство современных языков программирования принадлежат другой ветви — ветви Си-подобных языков, поэтому PL/pgSQL может показаться непривычным, избыточно многословным (характерное отличие — использование ключевых слов BEGIN и END вместо фигурных скобок). Впрочем, этот синтаксис отлично сочетается с синтаксисом SQL.

https://postgrespro.ru/docs/postgresql/16/plpgsql-overview

Структура блока

Метка блока

Объявления переменных

- область действия — блок
- область видимости может перекрываться вложенными блоками, но можно использовать метку блока
- любые типы SQL, ссылки на типы объектов (%TYPE)

Операторы

- управляющие конструкции
- операторы SQL, кроме служебных

Обработка исключительных ситуаций

Операторы языка PL/pgSQL объединяются в блоки. В структуре блока можно выделить:

- Необязательную метку, которую можно использовать для устранения неоднозначности в именовании.
- Необязательную секцию, предназначенную для *объявления* локальных переменных и курсоров. В качестве типа можно указать любой тип, определенный в SQL. Можно также сослаться на тип столбца таблицы с помощью конструкции %TYPE.
- Основную секцию исполнения, в которой располагаются *операторы*.
- Необязательную секцию *обработки исключительных ситуаций.*

В качестве операторов можно использовать как команды PL/pgSQL, так и большинство команд SQL, то есть этих два языка интегрированы практически бесшовно. Нельзя использовать служебные команды SQL, такие, как VACUUM. А команды управления транзакциями (например, COMMIT, ROLLBACK) допускаются только в процедурах.

В качестве оператора может выступать и другой (вложенный) PL/pgSQL-блок.

https://postgrespro.ru/docs/postgresql/16/plpgsql-structure

https://postgrespro.ru/docs/postgresql/16/plpgsql-declarations#PLPGSQL-DECLARATION-TYPE

Анонимные блоки

Разовое выполнение процедурного кода

- без создания хранимой подпрограммы
- без возможности передать параметры
- без возможности вернуть значение

Оператор DO языка SQL

Для использования PL/pgSQL не обязательно создавать подпрограммы. Код на PL/pgSQL можно оформить и выполнить как анонимный блок, при помощи SQL-команды DO.

Эту команду можно использовать с разными серверными языками, но если язык не указать явно, то будет использоваться PL/pgSQL.

Код анонимного блока не сохраняется на сервере. Также нет возможности передать в анонимный блок параметры или вернуть из него значение. Хотя косвенно это можно сделать, например, через таблицы.

https://postgrespro.ru/docs/postgresql/16/sql-do

Анонимные блоки

Общая структура блока PL/pgSQL:

```
<<метка>>
DECLARE
    -- объявления переменных
BEGIN
    -- операторы
EXCEPTION
    -- обработка ошибок
END метка;
```

- Все секции, кроме секции операторов, являются необязательными.

Минимальный блок PL/pgSQL-кода:

```
=> DO $$
BEGIN
    -- сами операторы могут и отсутствовать
END
$$;
```

```
DO
```

Вариант программы Hello, World!

```
=> DO $$
DECLARE
    -- Это однострочный комментарий.
    /* А это — многострочный.
       После каждого объявления ставится знак ';'.
       Этот же знак ставится после каждого оператора.
    */
    foo text;
    bar text := 'World'; -- также допускается = или DEFAULT
BEGIN
    foo := 'Hello'; -- это присваивание
    RAISE NOTICE '%, %!', foo, bar; -- вывод сообщения
END
$$;
```

```
NOTICE:  Hello, World!
DO
```

- После BEGIN точка с запятой не ставится!

Переменные могут иметь модификаторы:

- CONSTANT — значение переменной не должно изменяться после инициализации;
- NOT NULL — не допускается неопределенное значение.

```
=> DO $$
DECLARE
    foo integer NOT NULL := 0;
    bar CONSTANT text := 42;
BEGIN
    bar := bar + 1; -- ошибка
END
$$;
```

```
ERROR:  variable "bar" is declared CONSTANT
LINE 6:     bar := bar + 1; -- ошибка
            ^
```

Пример вложенных блоков. Переменная во внутреннем блоке перекрывает переменную из внешнего блока, но с помощью меток можно обратиться к любой из них:

```
=> DO $$
<<outer_block>>
DECLARE
    foo text := 'Hello';
BEGIN
    <<inner_block>>
    DECLARE
        foo text := 'World';
    BEGIN
        RAISE NOTICE '%, %!', outer_block.foo, inner_block.foo;
        RAISE NOTICE 'Без метки — внутренняя переменная: %', foo;
    END inner_block;
END outer_block
$$;
```

```
NOTICE:  Hello, World!
NOTICE:  Без метки — внутренняя переменная: World
DO
```

Подпрограммы PL/pgSQL

Заголовок подпрограммы не зависит от языка

- имя, входные и выходные параметры
- для функций: возвращаемое значение, категория изменчивости

Указание LANGUAGE plpgsql

Возврат значений

- оператор RETURN
- присвоение значений выходным (INOUT, OUT) параметрам

Мы уже познакомились с хранимыми функциями и процедурами на примере языка SQL. Большинство рассмотренных вопросов, связанных с созданием и управлением подпрограммами, относится и к подпрограммам на PL/pgSQL:

- создание, изменение, удаление подпрограмм;
- расположение в системном каталоге (pg_proc);
- параметры;
- возвращаемое значение и категории изменчивости (для функций);
- перегрузка и полиморфизм;
- и т. д.

Если в SQL подпрограмма возвращала значение, выданное последним SQL-оператором, то в подпрограммах на PL/pgSQL требуется либо присваивать возвращаемые значения формальным INOUT- и OUT-параметрам, либо (для функций) использовать специальный оператор RETURN.

Подпрограммы PL/pgSQL

Пример функции, возвращающей значение с помощью оператора RETURN:

```
=> CREATE FUNCTION sqr_in(IN a numeric) RETURNS numeric
AS $$
BEGIN
    RETURN a * a;
END
$$ LANGUAGE plpgsql IMMUTABLE;
```

```
CREATE FUNCTION
```

Та же функция, но с OUT-параметром. Возвращаемое значение присваивается параметру:

```
=> CREATE FUNCTION sqr_out(IN a numeric, OUT retval numeric)
AS $$
BEGIN
    retval := a * a;
END
$$ LANGUAGE plpgsql IMMUTABLE;
```

```
CREATE FUNCTION
```

Та же функция, но с INOUT-параметром. Такой параметр используется и для принятия входного значения, и для возврата значения функции:

```
=> CREATE FUNCTION sqr_inout(INOUT a numeric)
AS $$
BEGIN
    a := a * a;
END
$$ LANGUAGE plpgsql IMMUTABLE;
```

```
CREATE FUNCTION
```

```
=> SELECT sqr_in(3), sqr_out(3), sqr_inout(3);
```

```
 sqr_in | sqr_out | sqr_inout
--------+---------+-----------
      9 |       9 |         9
(1 row)
```

Условные операторы

IF

стандартный условный оператор

CASE

похож на CASE языка SQL, но не возвращает значение

Внимание: трехзначная логика

условие должно быть истинно; false и NULL не подходят

PL/pgSQL предлагает два условных оператора: IF и CASE.

Первый — совершенно стандартный оператор, имеющийся во всех языках.

Второй похож на конструкцию CASE в языке SQL, но это именно оператор, он не возвращает значение. Это некий аналог оператора switch в C или Java.

Важно всегда помнить о том, что логические выражения в SQL (и, следовательно, в PL/pgSQL) могут принимать *три* значения: true, false и NULL. Условие срабатывает, только когда оно истинно, и не срабатывает, когда ложно *или не определено*. Это одинаково верно и для условий WHERE в SQL, и для условных операторов PL/pgSQL.

https://postgrespro.ru/docs/postgresql/16/plpgsql-control-structures#PLPGSQL-CONDITIONALS

Условные операторы

Общий вид оператора IF:

```
IF условие THEN
    -- операторы
ELSIF условие THEN
    -- операторы
ELSE
    -- операторы
END IF;
```

- Секция ELSIF может повторяться несколько раз, а может отсутствовать.
- Секция ELSE может отсутствовать.
- Выполняются операторы, соответствующие первому истинному условию.
- Если ни одно из условий не истинно, выполняются операторы ELSE (если есть).

Пример функции, использующей условный оператор для форматирования номера телефона. Функция возвращает значение составного типа:

```
=> CREATE FUNCTION fmt (IN phone text, OUT code text, OUT num text)
AS $$
BEGIN
    IF phone ~ '^[0-9]*$' AND length(phone) = 10 THEN
        code := substr(phone,1,3);
        num  := substr(phone,4);
    ELSE
        code := NULL;
        num  := NULL;
    END IF;
END
$$ LANGUAGE plpgsql IMMUTABLE;
```

```
CREATE FUNCTION
```

```
=> SELECT fmt('8122128506');
```

```
      fmt
---------------
 (812,2128506)
(1 row)
```

Общий вид оператора CASE (первый вариант — по условию):

```
CASE
    WHEN условие THEN
        -- операторы
    ELSE
        -- операторы
END CASE;
```

- Секция WHEN может повторяться несколько раз.
- Секция ELSE может отсутствовать.
- Выполняются операторы, соответствующие первому истинному условию.
- Если ни одно из условий не истинно, выполняются операторы ELSE (отсутствие ELSE в таком случае — ошибка).

Пример использования:

```
=> DO $$
DECLARE
    code text := (fmt('8122128506')).code;
BEGIN
    CASE
        WHEN code IN ('495','499') THEN
            RAISE NOTICE '% — Москва', code;
        WHEN code = '812' THEN
            RAISE NOTICE '% — Санкт-Петербург', code;
        WHEN code = '384' THEN
            RAISE NOTICE '% — Кемеровская область', code;
        ELSE
            RAISE NOTICE '% — Прочие', code;
    END CASE;
END

$$;

NOTICE:  812 — Санкт-Петербург
DO
```

Общий вид оператора CASE (второй вариант — по выражению):

```
CASE выражение
    WHEN значение, ... THEN
        -- операторы
    ELSE
        -- операторы
END CASE;
```

- Секция WHEN может повторяться несколько раз.
- Секция ELSE может отсутствовать.
- Выполняются операторы, соответствующие первому истинному условию «выражение = значение».
- Если ни одно из условий не истинно, выполняются операторы ELSE (отсутствие ELSE в таком случае — ошибка).

При однотипных условиях эта форма CASE может оказаться компактней:

```
=> DO $$
DECLARE
    code text := (fmt('8122128506')).code;
BEGIN
    CASE code
        WHEN '495', '499' THEN
            RAISE NOTICE '% — Москва', code;
        WHEN '812' THEN
            RAISE NOTICE '% — Санкт-Петербург', code;
        WHEN '384' THEN
            RAISE NOTICE '% — Кемеровская область', code;
        ELSE
            RAISE NOTICE '% — Прочие', code;
    END CASE;
END
$$;

NOTICE:  812 — Санкт-Петербург
DO
```

Циклы

Цикл FOR по диапазону чисел

Цикл WHILE с предусловием

Бесконечный цикл

Цикл может иметь метку, как блок

Управление

- выход из цикла (EXIT)
- переход на новую итерацию (CONTINUE)

Для повторного выполнения набора операторов PL/pgSQL предлагает несколько видов циклов:

- цикл FOR по диапазону целых чисел;
- цикл WHILE с предусловием;
- бесконечный цикл.

Цикл представляет собой разновидность блока и может иметь собственную метку.

Выполнением цикла можно дополнительно управлять, инициируя переход на новую итерацию или выход из цикла.

https://postgrespro.ru/docs/postgresql/16/plpgsql-control-structures#PLPGSQL-CONTROL-STRUCTURES-LOOPS

Цикл FOR может работать не только по диапазону чисел, но и по результатам выполнения запроса и по массивам. Эти варианты цикла FOR будут рассмотрены в следующих темах.

Циклы

В PL/pgSQL все циклы используют общую конструкцию:

```
LOOP
    -- операторы
END LOOP;
```

К ней может добавляться заголовок, определяющий условие выхода из цикла.

Цикл по диапазону FOR повторяется, пока счетчик цикла пробегает значения от нижней границы до верхней. С каждой итерацией счетчик увеличивается на 1 (но инкремент можно изменить в необязательной фразе BY).

```
FOR имя IN низ .. верх BY инкремент
LOOP
    -- операторы
END LOOP;
```

- Переменная, выступающая счетчиком цикла, объявляется неявно и существует только внутри блока LOOP — END LOOP.

При указании REVERSE значение счетчика на каждой итерации уменьшается, а нижнюю и верхнюю границы цикла нужно поменять местами:

```
FOR имя IN REVERSE верх .. низ BY инкремент
LOOP
    -- операторы
END LOOP;
```

Пример использования цикла FOR — функция, "переворачивающая" строку:

```
=> CREATE FUNCTION reverse_for (line text) RETURNS text
AS $$
DECLARE
    line_length CONSTANT int := length(line);
    retval text := '';
BEGIN
    FOR i IN 1 .. line_length
    LOOP
        retval := substr(line, i, 1) || retval;
    END LOOP;
    RETURN retval;
END
$$ LANGUAGE plpgsql IMMUTABLE STRICT;
```

```
CREATE FUNCTION
```

Напомним, что строгая (STRICT) функция немедленно возвращает NULL, если хотя бы один из входных параметров не определен. Тело функции при этом не выполняется.

Цикл WHILE выполняется до тех пор, пока истинно условие:

```
WHILE условие
LOOP
    -- операторы
END LOOP;
```

Та же функция, обращающая строку, с использованием цикла WHILE:

```
=> CREATE FUNCTION reverse_while (line text) RETURNS text
AS $$
DECLARE
    line_length CONSTANT int := length(line);
    i int := 1;
    retval text := '';
BEGIN
    WHILE i <= line_length
    LOOP
        retval := substr(line, i, 1) || retval;
        i := i + 1;
    END LOOP;
    RETURN retval;
END
$$ LANGUAGE plpgsql IMMUTABLE STRICT;
```

```
CREATE FUNCTION
```

Цикл LOOP без заголовка выполняется бесконечно. Для выхода используется оператор EXIT:

```
EXIT метка WHEN условие;
```

- Метка необязательна; если не указана, будет прерван самый вложенный цикл.
- Фраза WHEN также необязательна; при отсутствии цикл прерывается безусловно.

Пример использования цикла LOOP:

```
=> CREATE FUNCTION reverse_loop (line text) RETURNS text
AS $$
DECLARE
    line_length CONSTANT int := length(reverse_loop.line);
    i int := 1;
    retval text := '';
BEGIN
    <<main_loop>>
    LOOP
        EXIT main_loop WHEN i > line_length;
        retval := substr(reverse_loop.line, i,1) || retval;
        i := i + 1;
    END LOOP;
    RETURN retval;
END
$$ LANGUAGE plpgsql IMMUTABLE STRICT;
```

```
CREATE FUNCTION
```

- Тело функции помещается в неявный блок, метка которого совпадает с именем функции. Поэтому к параметрам можно обращаться как «имя_функции.параметр».

Убедимся, что все функции работают правильно:

```
=> SELECT reverse_for('главрыба') as "for",
          reverse_while('главрыба') as "while",
          reverse_loop('главрыба') as "loop";
```

```
   for    |  while   |   loop
----------+----------+----------
 абырвалг | абырвалг | абырвалг
(1 row)
```

Замечание. В PostgreSQL есть встроенная функция reverse.

Иногда бывает полезен оператор CONTINUE, начинающий новую итерацию цикла:

```
=> DO $$
DECLARE
    s integer := 0;
BEGIN
    FOR i IN 1 .. 100
    LOOP
        s := s + i;
        CONTINUE WHEN mod(i, 10) != 0;
        RAISE NOTICE 'i = %, s = %', i, s;
    END LOOP;
END
$$;
```

```
NOTICE:  i = 10, s = 55
NOTICE:  i = 20, s = 210
NOTICE:  i = 30, s = 465
NOTICE:  i = 40, s = 820
NOTICE:  i = 50, s = 1275
NOTICE:  i = 60, s = 1830
NOTICE:  i = 70, s = 2485
NOTICE:  i = 80, s = 3240
NOTICE:  i = 90, s = 4095
NOTICE:  i = 100, s = 5050
DO
```

Вычисление выражений

Любое выражение вычисляется в контексте SQL

- выражение автоматически преобразуется в запрос
- запрос подготавливается
- переменные PL/pgSQL подставляются как параметры

Особенности

- можно использовать все возможности SQL, включая подзапросы
- невысокая скорость выполнения,
  хотя разобранный запрос (и, возможно, план запроса) кешируются
- неоднозначности при разрешении имен требуют внимания

Выражения, встречающиеся в PL/pgSQL-коде, вычисляются с помощью SQL-запросов к базе данных. Интерпретатор сам строит нужный запрос, формируя оператор вида SELECT <expr> и заменяя переменные PL/pgSQL на параметры, затем подготавливает его (при этом разобранный запрос кешируется, как это обычно и происходит с подготовленными операторами), после чего планирует и выполняет.

С одной стороны, такой подход не способствует скорости работы PL/pgSQL, зато обеспечивает теснейшую интеграцию с SQL. Фактически, в выражениях можно использовать любые возможности SQL без ограничений, включая вызов встроенных и пользовательских функций, выполнение подзапросов и т. п.

Начиная с версии PostgreSQL 14 оптимизировано выполнение простых выражений (как минимум не содержащих обращений к таблицам) — такие выражения обрабатываются синтаксическим анализатором сервера напрямую, без обращений к планировщику.

https://postgrespro.ru/docs/postgresql/16/plpgsql-expressions

Вычисление выражений

Выражения в PL/pgSQL выполняются SQL-движком. Таким образом, в PL/pgSQL доступно ровно то, что доступно в SQL. Например, если в SQL можно использовать конструкцию CASE, то точно такая же конструкция будет работать и в коде на PL/pgSQL (в качестве выражения; не путайте с оператором CASE ... END CASE, который есть только в PL/pgSQL):

```
=> DO $$
BEGIN
    RAISE NOTICE '%', CASE 2+2 WHEN 4 THEN 'Все в порядке' END;
END
$$;
```

```
NOTICE:  Все в порядке
DO
```

В выражениях можно использовать и подзапросы:

```
=> DO $$
BEGIN
    RAISE NOTICE '%', (
        SELECT code
        FROM (VALUES (1, 'Раз'), (2, 'Два')) t(id, code)
        WHERE id = 1
    );
END
$$;
```

```
NOTICE:  Раз
DO
```

Еще пример с вычислением выражения в PL/pgSQL — сколько функций «обращения» строк у нас в результате получилось?

```
=> DO $$
DECLARE
  s integer;
BEGIN
  s := count(*) FROM pg_proc WHERE proname LIKE 'reverse%';
  RAISE NOTICE 'Всего функций "reverse" : %', s;
END
$$;
```

```
NOTICE:  Всего функций "reverse" : 4
DO
```

Итоги

PL/pgSQL — доступный по умолчанию, интегрированный с SQL, удобный и простой в использовании язык

Управление подпрограммами на PL/pgSQL не отличается от работы с подпрограммами на других языках

DO — команда SQL для выполнения анонимного блока

Переменные PL/pgSQL могут использовать любые типы SQL

Язык поддерживает обычные управляющие конструкции, такие как условные операторы и циклы